

Reviewer Pack — Minimal Rank of Quasi-crystalline Sets over Monotone Circuit...

Entry 7202c9813218 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 01:34:42 UTC

Minimal Rank of Quasi-crystalline Sets over Monotone Circuit Size

Entry ID: 7202c9813218 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 01:34:42 UTC

1. Conjecture

Field A (mathematical branch): Quasi-crystallography **Field B** (complexity object): Complexity Theory: Monotone Circuit Complexity

Statement:

[‘For every monotone Boolean function f with n variables, define a quasi-crystalline set Q_f as the closure of the set of all integer points $(a_1, \dots, a_n)$ in $\mathbb{Z}^n$ such that $f(a_1, \dots, a_n) = 1$. Let $r(Q_f)$ be the minimal rank of Q_f over any finite field F_q with q elements. Then, for any fixed q , $r(Q_f)$ is upper bounded by $O(\log n)$ and lower bounded by $\Omega(n)$.’, ‘Furthermore, there exists a constructive mapping from a monotone circuit C to a quasi-crystalline set Q_C such that the size of Q_C is polynomial in the size of C .’, ‘For every monotone circuit C with m gates and n variables, the minimal rank of its associated quasi-crystalline set Q_C satisfies $r(Q_C) = \Theta(m \log n)$.’]

Rationale (proposer’s reasoning):

[‘Quasi-crystallography provides a framework for studying discrete structures that exhibit some degree of periodicity. By mapping monotone circuits to quasi-crystalline sets, we can leverage the algebraic properties of these sets to gain insights into circuit complexity.’, ‘The relationship between the rank of a quasi-crystalline set and its associated circuit size could provide a new perspective on proving lower bounds for monotone circuits.’, ‘This conjecture builds upon previous work in both quasi-crystallography and circuit complexity, suggesting a potential new approach to understanding the fundamental limits of computation.’]

Taxonomy category: Quasi-crystallography (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e10b29e538a1a733

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a monotone circuit C with m gates and n variables, if the associated quasi-crystalline set Q_C has a minimal rank $r(Q_C)$ within the bounds $O(\log n)$ or $\Omega(n)$, or if there exists at least one seed where $r(Q_C)$ is between $O(\log n)$ and $\Omega(n)$, then the conjecture is supported. If any seed yields $r(Q_C)$ outside these bounds, the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	SAFE
NATURAL_PROOFS	SAFE	1.00	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "quasi-crystallography" AND "monotone circuit complexity" AND minimal rank" - "minimal rank of quasi-crystalline sets" AND monotone circuits" - "polynomial size mapping" FROM monotone circuits TO quasi-crystalline sets"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_monotone_circuit(n, m):
        # Simplified generation for demonstration purposes
        circuit = []
        for _ in range(m):
            gate_type = random.choice(['AND', 'OR'])
            inputs = random.sample(range(1, n+1), 2)
            circuit.append((gate_type, inputs))
        return circuit

    def construct_quasi_crystalline_set(circuit):
        # Simplified construction for demonstration purposes
        Q_C = set()
        for a in range(-10, 11): # Example range for simplicity
            for b in range(-10, 11):
                if evaluate_circuit(circuit, [a, b]):
                    Q_C.add((a, b))
        return Q_C
```

```

def evaluate_circuit(circuit, inputs):
    stack = []
    for gate_type, inputs in circuit:
        a, b = inputs
        if gate_type == 'AND':
            result = stack[a-1] and stack[b-1]
        elif gate_type == 'OR':
            result = stack[a-1] or stack[b-1]
        stack.append(result)
    return stack[-1]

def compute_rank(Q_C):
    # Simplified rank computation for demonstration purposes
    if not Q_C:
        return 0
    max_x = max(x for x, y in Q_C)
    max_y = max(y for x, y in Q_C)
    min_x = min(x for x, y in Q_C)
    min_y = min(y for x, y in Q_C)
    rank = (max_x - min_x + 1) * (max_y - min_y + 1)
    return rank

n = random.randint(5, 40)
m = random.randint(2*n, 3*n)
circuit = generate_monotone_circuit(n, m)
Q_C = construct_quasi_crystalline_set(circuit)
r_Q_C = compute_rank(Q_C)

return {
    "metric_name": "Minimal Rank",
    "metric_value": r_Q_C,
    "instances_tested": 1,
    "conjecture_holds": r_Q_C <= math.log(n) and r_Q_C >= n,
    "counterexample": "" if r_Q_C in range(math.ceil(math.log(n)), n+1) else f"r(Q_C) = {r_Q_C}"
}

if __name__ == "__main__":
    import sys
    seeds = list(map(int, sys.argv[1:])) or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={result['counterexample']} first_failing_seed={first_failing_seed}")
    else:

```

```
print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1e7b97ec.py", line 81, in <module>
    result = run_trial(seed)
            ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1e7b97ec.py", line 64, in run_trial
    Q_C = construct_quasi_crystalline_set(circuit)
        ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1e7b97ec.py", line 35, in construct_qua
    if evaluate_circuit(circuit, [a, b]):
    ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1e7b97ec.py", line 46, in evaluate_circ
    result = stack[a-1] or stack[b-1]
            ~~~~~
```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data that could confirm or falsify the conjecture.
| next: Re-run the test with proper error handling to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13194
2	preregistration	ollama_remote	glm4:latest	0	0	6177
3	novelty	ollama_remote	glm4:latest	0	0	4785
4	novelty	ollama_remote	glm4:latest	0	0	5618
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20061
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12663
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17307
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9772
9	judge	ollama_remote	glm4:latest	0	0	7879

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 97456 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/7202c9813218.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/7202c9813218.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/7202c9813218.tar.gz (if generated)